МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РЕСПУБЛИКИ БАШКОРТОСТАН
ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ УФИМСКИЙ КОЛЛЕДЖ СТАТИСТИКИ, ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

SOLID–принципы

СТУДЕНТ ГРУППЫ 22П-1: СЕМЕНОВ М.В.

ПРЕПОДАВАТЕЛЬ: НАБИЕВА Л.Р.

УФА – 2026

Цели

1. Узнать что такое принципы проектирования;
2. Узнать что такое SOLID–принципы;
3. Узнать какие принципы входят в SOLID, их отличия и примеры на практике.

Что такое принципы проектирования?

Принципы проектирования охватывают фундаментальные правила создания надежных систем, направленные на снижение сложности, гибкость повторное использование и т.п. К основным видам относятся архитектурные паттерны (SOLID, GRASP) и подходы (KISS, DRY, YAGNI).

SOLID (объектно-ориентированное проектирование)

Принципы SOLID – это пять правил объектно-ориентированного проектирования, которые помогают делать код понятным, гибким и легким в поддержке. Их цель – избежать «хрупкости» кода, когда изменение одной части ломает другую, и упростить развитие проектов. Они позволяют создавать архитектуру, которую легко читать и дорабатывать.

Виды SOLID – принципов:

1. S – Single Responsibility Principle(SRP);
2. O – Open/Closed Principle(OCP);
3. L – Liskov Substitution Principle(LSP);
4. I – Interface Segregation Principle(ISP);
5. D – Dependency Inversion Principle(DIP).

Single Responsibility Principle(SRP)

SRP (Принцип единственной ответственности): Класс должен иметь только одну причину для изменения, то есть отвечать только за одну конкретную задачу. Но если есть схожие для изменения вещи они должны храниться в одном месте.

SRP не только требует "дробить пока дробится", но и не перестараться — "не раздробить сцепленные вещи". Не усложнять без надобности.

Пример не соблюдения SRP(C#)

```
public class Calculate
{
    Ссылок: 0
    public void AdditionMethod()
    {
        Console.Write("Введите первое число: ");
        string firstNumberString = Console.ReadLine();
        Console.Write("Введите второе число: ");
        string secondNumberString = Console.ReadLine();

        Regex regex = new Regex(@"^\d+$");

        if (!regex.IsMatch(firstNumberString) || !regex.IsMatch(secondNumberString))
        {
            throw new Exception("Не корректные данные!");
        }

        int firstNumber = Convert.ToInt32(firstNumberString);
        int secondNumber = Convert.ToInt32(secondNumberString);

        int sum = firstNumber + secondNumber;
        Console.WriteLine(sum);
    }
}
```

Пример соблюдения SRP(C#)

```
public class DataReader
{
    Ссылок: 1
    public string[] GetInputData()
    {
        Console.Write("Введите первое число: ");
        string firstNumberString = Console.ReadLine();
        Console.Write("Введите второе число: ");
        string secondNumberString = Console.ReadLine();
        return new string[] { firstNumberString, secondNumberString };
    }
}
```

```
public class DataAddition
{
    Ссылок: 1
    public int AdditionInt(int[] data)
    {
        return data[0] + data[1];
    }
}
```

```
public class DataPrint
{
    Ссылок: 1
    public void WriteConsole(int data)
    {
        Console.WriteLine(data);
    }
}
```

```
public class DataValidator
{
    Ссылок: 0
    public bool IsValid(string[] data)
    {
        Regex regex = new Regex(@"^\d+$");

        if (!regex.IsMatch(data[0]) || !regex.IsMatch(data[1]))
            return false;

        return true;
    }
}
```

```
public class DataConverter
{
    Ссылок: 1
    public int[] ParseInt(string[] data)
    {
        return new int[] { Convert.ToInt32(data[0]), Convert.ToInt32(data[1]) };
    }
}
```

Пример соблюдения SRP(C#)

```
public class CalculateSRP
{
    Ссылок: 2
    public DataReader Reader {  get; set; }
    Ссылок: 2
    public DataValidator Validator {  get; set; }
    Ссылок: 2
    public DataConverter Converter {  get; set; }
    Ссылок: 2
    public DataAddition Addition {  get; set; }
    Ссылок: 2
    public DataPrint Print {  get; set; }
    Ссылок: 0
    public CalculateSRP(DataReader reader, DataValidator validator, DataConverter converter, DataAddition addition, DataPrint print)
    {
        this.Reader = reader;
        this.Validator = validator;
        this.Converter = converter;
        this.Addition = addition;
        this.Print = print;
    }

    Ссылок: 0
    public void Process()
    {
        string[] data = Reader.GetInputData();
        if (Validator.IsValid(data))
        {
            throw new Exception("Не корректные данные!");
        }
        int[] dataInt = Converter.ParseInt(data);
        int sum = Addition.AdditionInt(dataInt);
        Print.WriteConsole(sum);
    }
}
```

Open/Closed Principle(OCP)

OCP(Принцип открытости/закрытости): программные сущности (классы, модули, функции) должны быть открыты для расширения, но закрыты для изменения. Это означает, что новый функционал добавляется путем создания нового кода, а не редактирования уже существующего и протестированного.

Пример не соблюдения OCP(C#)

```
public class Cat
{
    Ссылок: 0
    public void MakeSound()
    {
        Console.WriteLine("Мяу");
    }
}
```

Пример соблюдения OCP(C#)

```
public interface IAnimal
{
    Ссылок: 0
    void MakeSound();
}
```

```
public class CatOCP : IAnimal
{
    Ссылок: 1
    public void MakeSound()
    {
        Console.WriteLine("Мяу");
    }
}
```

Liskov Substitution Principle(LSP)

LSP(Принцип подстановки Барбары Лисков): функции, использующие базовый тип, должны уметь работать с его подтипами, не зная об этом. Подклассы не должны нарушать поведение базового класса.

Главные признаки нарушения LSP:

1. Пустые методы: Вы переопределили метод родителя, но он ничего не делает (потому что подклассу это не нужно);
2. Выбрасывание исключений там, где их нет у родителя;
3. Проверка типа: Если внутри функции вам приходится проверять, какой именно подтип вам пришел, чтобы выполнить разные действия – это верный признак нарушения LSP.

Пример не соблюдения LSP(C#)

```
public class Square : Rectangle
{
    Ссылок: 1
    public Square(int width) : base(width, width)
    {

    }

    Ссылок: 3
    public override void SetWidth(int value)
    {
        this.Width = value;
        this.Height = value;
    }

    Ссылок: 3
    public override void SetHeight(int value)
    {
        this.Height = value;
        this.Width = value;
    }
}
```

```
public class Rectangle
{
    Ссылок: 7
    private int Width { get; set; }
    Ссылок: 7
    private int Height { get; set; }

    Ссылок: 0
    public int GetWidth()
    {
        return this.Width;
    }

    Ссылок: 0
    public int GetHeight()
    {
        return this.Height;
    }

    Ссылок: 0
    public void SetWidth(int value)
    {
        this.Width = value;
    }

    Ссылок: 0
    public void SetHeight(int value)
    {
        this.Height = value;
    }

    Ссылок: 2
    public Rectangle(int width, int height)
    {
        this.Width = width;
        this.Height = height;
    }

    Ссылок: 2
    public int GetArea()
    {
        return Width * Height;
    }
}
```

Пример соблюдения LSP(C#)

```
public interface IShape
{
    Ссылок: 2
    int GetArea();
}
```

```
public class SquareLSP : IShape
{
    Ссылок: 5
    public int Side { get; set; }
    Ссылок: 0
    public SquareLSP(int side)
    {
        this.Side = side;
    }
    Ссылок: 1
    public int GetArea()
    {
        return Side * Side;
    }
    Ссылок: 0
    public int GetSide()
    {
        return this.Side;
    }

    Ссылок: 0
    public void SetSide(int value)
    {
        this.Side = value;
    }
}
```

```
public class RectangleLSP : IShape
{
    Ссылок: 4
    private int Width { get; set; }
    Ссылок: 4
    private int Height { get; set; }
    Ссылок: 0
    public RectangleLSP(int width, int height)
    {
        this.Width = width;
        this.Height = height;
    }

    Ссылок: 1
    public int GetArea()
    {
        return Width * Height;
    }
    Ссылок: 0
    public int GetWidth()
    {
        return this.Width;
    }

    Ссылок: 0
    public int GetHeight()
    {
        return this.Height;
    }

    Ссылок: 0
    public void SetWidth(int value)
    {
        this.Width = value;
    }

    Ссылок: 0
    public void SetHeight(int value)
    {
        this.Height = value;
    }
}
```

Interface Segregation Principle(ISP)

Принцип разделения интерфейса (ISP): «Клиенты не должны зависеть от методов, которые они не используют». Вместо одного толстого интерфейса нужно создавать несколько маленьких, специализированных, чтобы код оставался гибким, понятным и его было легко поддерживать.

Интерфейсы должны быть максимально узкими (сплоченными), объединяя методы только для одной конкретной функциональности.

Пример не соблюдения ISP(C#)

```
public class VESTA : ICar
{
    Ссылок: 1
    public void ABS()
    {
        Console.WriteLine("Включение ABS");
    }
    Ссылок: 1
    public void ESP()
    {
        Console.WriteLine("Включение ESP");
    }
    Ссылок: 1
    public void LKA() { }
}
```

```
public interface ICar
{
    Ссылок: 1
    void ABS();
    Ссылок: 1
    void ESP();
    Ссылок: 1
    void LKA();
}
```

```
public class BMW_f30_320i : ICar
{
    Ссылок: 1
    public void ABS()
    {
        Console.WriteLine("Включение ABS");
    }
    Ссылок: 1
    public void ESP()
    {
        Console.WriteLine("Включение ESP");
    }
    Ссылок: 1
    public void LKA()
    {
        Console.WriteLine("Включение LKA");
    }
}
```

Пример соблюдения ISP(C#)

```
public interface IABS
{
    Ссылок: 0
    void ABS();
}
```

```
public interface IESP
{
    void ESP();
}
```

```
public interface ILKA
{
    void LKA();
}
```

```
public class VESTA : IABS, IESP
{
    Ссылок: 1
    public void ABS()
    {
        Console.WriteLine("Включение ABS");
    }
    Ссылок: 1
    public void ESP()
    {
        Console.WriteLine("Включение ESP");
    }
}
```

```
public class BMW_f30_320i : IABS, IESP, ILKA
{
    Ссылок: 1
    public void ABS()
    {
        Console.WriteLine("Включение ABS");
    }
    Ссылок: 1
    public void ESP()
    {
        Console.WriteLine("Включение ESP");
    }
    Ссылок: 1
    public void LKA()
    {
        Console.WriteLine("Включение LKA");
    }
}
```

Dependency Inversion Principle(DIP)

Принцип инверсии зависимостей (DIP): модули верхних уровней не должны зависеть от модулей нижних уровней; оба типа модулей должны зависеть от абстракций (интерфейсов)а не деталей.

1. Модули верхних уровней не должны зависеть от модулей нижних уровней. Оба типа модулей должны зависеть от абстракций.
2. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

Пример не соблюдения DIP(C#)

```
public class Datadase
{
    Ссылок: 0
    public void SaveDatabase()
    {
        Console.WriteLine("Сохранение в базу данных.");
    }
}
```

```
public class File
{
    Ссылок: 0
    public void WriteFile()
    {
        Console.WriteLine("Запись в файл.");
    }
}
```

```
public class NotePad
{
    private readonly File File;

    Ссылок: 0
    public NotePad(File file)
    {
        this.File = file;
    }

    Ссылок: 0
    public void SaveNewNote()
    {
        Console.WriteLine("Создание новой заметки.");
        this.File.SaveDatabase();
    }
}
```

```
public class NotePad
{
    private readonly Datadase Datadase;

    Ссылок: 0
    public NotePad(Datadase datadase)
    {
        this.Datadase = datadase;
    }

    Ссылок: 0
    public void SaveNewNote()
    {
        Console.WriteLine("Создание новой заметки.");
        this.Datadase.SaveDatabase();
    }
}
```

Пример соблюдения DIP(C#)

```
public class FileDIP : IDatabase
{
    Ссылок: 1
    public void SaveDatabase()
    {
        Console.WriteLine("Запись в файл.");
    }
}
```

```
public class DatadaseDIP : IDatabase
{
    Ссылок: 1
    public void SaveDatabase()
    {
        Console.WriteLine("Сохранение в базу данных.");
    }
}
```

```
public class NotePadDIP
{
    private readonly IDatabase Database;

    Ссылок: 0
    public NotePadDIP(IDatabase database)
    {
        this.Database = database;
    }

    Ссылок: 0
    public void SaveNewNote()
    {
        Console.WriteLine("Создание новой заметки.");
        this.Database.SaveDatabase();
    }
}
```

Отличия принципов SOLID:

SRP (Принцип единственной ответственности): фокусируется на связности внутри класса не на его взаимодействии с другими.

OCP (Принцип открытости/закрытости): фокусируется на возможности добавления функционала без разрушения рабочего кода.

LSP (Принцип подстановки Барбары Лисков): фокусируется на правильном наследовании и поведении подтипов.

ISP (Принцип разделения интерфейсов): фокусируется на устранении ненужных зависимостей разбивая большие интерфейсы.

DIP (Принцип инверсии зависимостей): фокусируется на уменьшении связанности между компонентами.

Спасибо за внимание!